

Article

Bridge Points Guided Neural Motion Planning in Complex Environments with Narrow Passages

Songyi Dian ^{1,2} , Juntong Liu ¹ , Guofei Xiang ^{1,*}  and Xingxing You ¹ ¹ Department of Automation, College of Electrical Engineering, Sichuan University, Chengdu 610065, China² State Key Laboratory of Intelligent Construction and Healthy Operation and Maintenance of Deep Underground Engineering, Sichuan University, Chengdu 610065, China

* Correspondence: gfxiang@scu.edu.cn

Abstract

Motion and path planning are fundamental to intelligent robotic systems, enabling navigation. The objective is to generate collision-free trajectories in obstacle-rich configuration spaces (C-spaces) while meeting performance constraints. In environments with narrow passages planning becomes especially difficult, as feasible regions have low measure and are rarely reached by random sampling. Classical sampling-based planners are probabilistically complete but inefficient in such regions. Learning-based planners like MPNet offer fast inference but often produce infeasible paths in cluttered areas, requiring expensive postprocessing. To address this trade-off, we propose a hybrid framework that combines improved sampling, structural abstraction, and neural prediction. A modified bridge-test sampler applies directional perturbations and corridor checks to generate reliable narrow passage samples. These are clustered into a sparse set of representative bridge points, which serve as nodes in a global graph. At query time, a greedy heuristic search explores this graph, using a neural local segment generator to connect nodes. We validate the approach on 2D maze maps, 3D voxel environments, and a 12-DOF manipulator performing a plugging task inside a simulated nuclear steam generator. Across all tasks, our method significantly outperforms classical and learning-based baselines in terms of success rate and planning time in narrow-passage-dominated scenarios. The inclusion of the repair module, under relaxed assumptions, also allows the framework to retain a generalized form of probabilistic completeness.

Keywords: motion planning; narrow passages; bridge test; neural planner

Academic Editor: Zheng Chen

Received: 26 January 2026

Revised: 20 February 2026

Accepted: 26 February 2026

Published: 3 March 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Motion planning in environments with narrow passages remains a fundamental challenge in robotics because feasible solutions occupy a low-measure subset of the configuration space. Classical sampling-based planners such as Rapidly-exploring Random Trees (RRT) [1] and RRT* [2] explore the space through repeated random sampling and collision checking. They can be probabilistically complete under standard assumptions, yet their efficiency degrades sharply in cluttered scenes: most samples fall in irrelevant open regions and the probability of hitting passage entrances becomes extremely low. Bidirectional variants such as BiRRT [3] partially improve connectivity, but they still lack an explicit mechanism to concentrate computation on the critical bottleneck geometry.

In contrast, learning-based planners such as MPNet [4] leverage offline demonstrations to infer waypoints with low inference cost, effectively injecting a data-driven prior to speed

up long-horizon planning. However, this prior can be brittle near tight gaps: under distribution shift, predicted waypoints may miss the correct passage entrances or violate constraints, which then triggers costly post-hoc repairs (e.g., RRT-Connect [3]) and offsets the gain of fast inference. This contrast highlights a key gap: sampling-based methods are guarantee-driven but budget-inefficient in narrow passages, whereas learning-based methods are fast but may fail to generalize at bottlenecks.

Therefore, our goal is not to replace geometric heuristics with learning, but to factorize the long-horizon planning problem into two complementary sub-problems: (i) topology exposure, where an explicit geometric mechanism extracts reliable bottleneck evidence (bridge points) to reveal passage structure; and (ii) fast local connection, where a learned segment generator amortizes the cost of repeatedly connecting nearby states once the topology is exposed. The bounded repair and backtracking mechanism serves as a safety net that preserves feasibility and robustness when the learned connector fails in tight regions.

In this paper, we propose a hybrid path planning framework that integrates geometric structure awareness with neural segment generation, achieving both speed and robustness. Our method first applies an improved bridge sampling strategy that perturbs candidate points along orthogonal directions and filters them using a single-channel criterion, effectively concentrating samples within true narrow passages while eliminating spurious points near obstacle boundaries or open space. The remaining valid bridge points are then clustered using DBSCAN to form a sparse set of representative passage centroids, which serve as global nodes in a sparse planning graph.

Between these nodes, a lightweight neural segment generator predicts feasible local trajectories. If the connection fails, a bounded number of RRT-Connect repairs are attempted, and backtracking is triggered if necessary to preserve completeness. Unlike traditional graph-based planners, our method avoids exhaustive graph construction or enumeration of node permutations, and instead employs a greedy heuristic guided by distance and residual goal cost, significantly reducing inference time while maintaining solution reliability.

We validate our method across three representative domains: (1) 2D maze environments with varying corridor layouts, (2) 3D spatial tasks involving double-hole and obstructed barriers, and (3) steam generator maintenance scenarios involving a 12-DOF robotic manipulator in a confined operational space. Experimental results demonstrate that our bridge-guided neural planner consistently outperforms classical sampling methods, bidirectional planners, and purely learned generators in success rate, planning time, and path quality. Furthermore, we provide a theoretical analysis showing that the overall planner inherits probabilistic completeness from the sampling-based repair component under standard assumptions, i.e., the success probability approaches one as the planning budget increases. The main contributions of this work are:

- Bridge-guided topology exposure with higher-quality bottleneck evidence. Compared with the classic Bridge Test and obstacle-based narrow-passage samplers, we propose an improved bridge sampling strategy (orthogonal perturbation and single-channel filtering) that produces sparser yet more accurate passage evidence, reducing false positives in open regions and near obstacle boundaries. This yields a compact set of valid bridge points that better reveals passage topology under the same sampling budget.
- Sparse structural abstraction for global guidance. Unlike using dense passage samples directly or constructing exhaustive graphs, we cluster valid bridge points via DBSCAN and use the resulting centroids as sparse global beacons to guide long-horizon planning. This abstraction reduces the number of global nodes and mitigates redundant exploration, enabling efficient planning without enumerating node permutations.

- Learning-accelerated local connection with completeness-preserving repair. In contrast to purely data-driven local planners that may fail under distribution shift, we integrate a lightweight neural segment generator with bounded RRT-Connect repair and backtracking. The learned connector amortizes repeated local connections once topology is exposed, while the sampling-based repair serves as a safety net that preserves feasibility and robustness in tight regions and supports probabilistic completeness under standard assumptions.

2. Related Works

Sampling-based motion planning methods such as RRT [1] and PRM [5] remain effective in high-dimensional spaces, yet narrow passages correspond to low-measure feasible sets and therefore dominate failure cases. We review related efforts from four complementary perspectives: (i) narrow-passage biased sampling, (ii) tree/roadmap expansion strategies, (iii) learning-based acceleration, and (iv) other complementary paradigms. Throughout this section, we emphasize how these lines of work motivate our bridge-guided topology exposure and completeness-preserving hybrid design.

2.1. Narrow-Passage Biased Sampling

These methods reshape the sampling distribution to increase the probability of entering bottlenecks. Representative ideas include expansiveness-driven exploration such as EST [6], bottleneck-aware sampling guided by geometric preprocessing or distance-field reasoning [7,8], and adaptive/informed sampling that concentrates effort in promising cost regions [9]. Bridge Test [10] is a classical technique that probes narrow regions by sampling “bridge” configurations across obstacles; it has inspired many subsequent variants. Other distribution-shaping strategies include dynamic-domain ideas [11], density/entropy-guided resampling [12,13], and boundary retraction [14]. Learning can also be used to bias sampling by learning connectivity structure or manifolds to improve coverage in bottlenecks [15]. Different from prior samplers that mainly increase the quantity of bottleneck samples, our improved bridge strategy aims to produce sparser but higher-quality passage evidence (lower redundancy and fewer spurious samples in open space), which is crucial for constructing a compact global guidance structure.

2.2. Tree/Roadmap Expansion and Bottleneck Connectivity

A complementary line improves connectivity by modifying how trees/roadmaps expand near constrained regions. Multi-tree or local-tree schemes can seed auxiliary trees around hard-to-connect samples and later merge them to improve passage traversal [16–18]. Other approaches compute local geometry to guide expansion directions, e.g., obstacle-aware steering [19] or PCA-based principal passage direction estimation [20,21]. More recently, multilevel planning isolates bottlenecks as low-dimensional sections and stitches solutions to boost success in high-DOF narrow passages [22]. Some works also incorporate bridge-test or directional heuristics into RRT-style expansion to improve entrance discovery [23]. In contrast, we do not rely on exhaustive multi-tree growth or expensive geometric computation at every expansion step. Instead, we explicitly expose passage topology via bridge evidence and then solve long-horizon planning on a sparse set of passage centroids, delegating only short-range connections to a lightweight local connector.

2.3. Learning-Based Planning Acceleration

Learning-based planners attempt to amortize planning cost by predicting paths or local connections from data. MPNet [4] learns a latent environment representation and predicts coarse trajectories quickly, but may fail under distribution shift or in very tight corridors where accurate entrance discovery is required. Other methods learn feasibility or

guidance signals on discretized workspaces [24,25], or extract geometric features from point clouds (e.g., PointNet-style encoders) to model free-space structure [26]. Reinforcement learning has also been combined with sampling-based planners to learn extension policies in difficult environments [27]. Our method treats learning as a local accelerator rather than a standalone planner: once topology is exposed by geometric bridge evidence, a lightweight neural segment generator connects nearby states, and a sampling-based repair is invoked when learning fails in tight regions.

2.4. Other Complementary Paradigms

Optimization and decomposition-based approaches offer alternative perspectives on constrained planning, e.g., tightening collision constraints in a homotopy-based trajectory optimization framework [28], safe-box graph decomposition with convex smoothing for provably safe navigation [29], and enhanced variants combining goal biasing and smoothing for industrial narrow gaps [30]. Importance-sampling bidirectional planners also improve efficiency in specific domains such as micro-scale ducts [31–33]. These works highlight the broader trend of combining global structure reasoning with fast local solvers, which aligns with our hybrid design that couples sparse passage-graph guidance with learning-driven local connection and completeness-preserving repair.

3. Methodology

This section details the three core components of our bridge-guided path-planning framework, whose overall workflow is illustrated in Figure 1. Section A explains how potential narrow-passage key points are extracted from the configuration space: an improved bridge-sampling procedure, followed by DBSCAN clustering, yields a sparse yet representative set of bridge points. Section B introduces the learning-driven local segment generator, which predicts collision-free trajectory segments between successive bridge points and validates them through fast collision checking, thereby populating the edge set of the inter-segment connection graph. Section C focuses on the query phase, where a greedy expansion strategy heuristically orders bridge points and iteratively invokes the local planner; any failed link is repaired by RRT-Connect, and the concatenated path is finally smoothed to obtain the end-to-end trajectory.

3.1. Bridge Point Sampling and Representative Extraction

To efficiently identify potential narrow passage regions in the workspace, this paper proposes an improved bridge sampling strategy based on the traditional Bridge Test, applicable to both 2D and 3D spaces. The method begins by randomly sampling two points within obstacles. If the midpoint of the line segment connecting these two points lies in free space, it is regarded as a candidate bridge point. To filter out pseudo-bridge points located at open-space edges without meaningful passability, a directional constraint mechanism is introduced: local perturbation sampling is conducted along directions orthogonal to the original line segment. These perturbations are used to check for the presence of clear obstacle boundary features. A point is considered a valid bridge point only if there exists exactly one direction with free space among the orthogonal directions, indicating structural “single-channel traversability.” Figure 2 illustrates the effect: the classical Bridge Test (a) produces many spurious samples at concave corners, whereas the proposed test (b) confines accepted points to the true narrow-passage region.

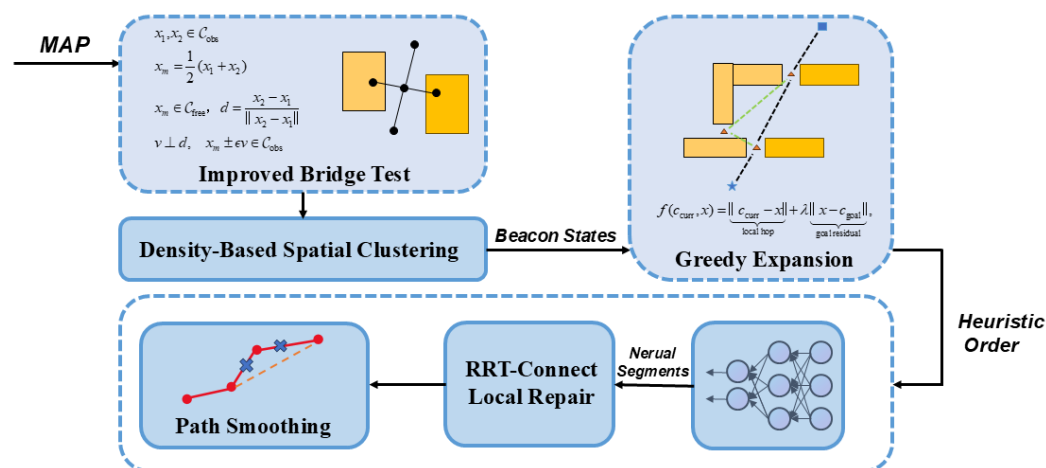


Figure 1. Bridge points guided neural motion planning pipeline. Given a map, Improved Bridge Test generates candidate configurations concentrated around narrow passages by orthogonal perturbation and validity checks. Density-Based Spatial Clustering (DBSCAN) groups valid candidates and outputs a sparse set of representative beacon states. Greedy Expansion orders and selects beacons to expose the long-horizon passage topology without exhaustive graph enumeration. A lightweight network predicts Neural Segments between successive beacons; if a predicted segment is infeasible, RRT-Connect Local Repair is invoked to recover feasibility. Finally, the concatenated path is refined by Path Smoothing to produce the final path.

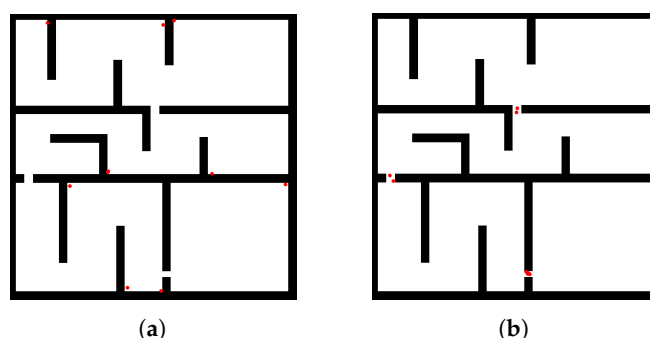


Figure 2. Comparison of bridge-point sampling behaviors in a narrow-passage scene. Each red dot denotes an accepted bridge midpoint after collision checks. (a) The classic Bridge Test tends to accept midpoints near concave corners, producing many false positives that do not represent the passage interior. (b) Our improved variant suppresses these corner samples and concentrates accepted points along the geometrical centerline of the true narrow corridor, yielding a cleaner and more informative bottleneck evidence set for subsequent clustering.

Through multiple rounds of sampling and filtering, a high-quality set of candidate bridge points is obtained. Considering that the candidate bridge points may be densely distributed and redundant, the Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm is further employed to perform clustering on the candidate set. The centroid of each cluster is selected as the representative bridge point, resulting in a sparse, well-covered set of representative bridge points. This set serves as the core node structure of the subsequent path graph, laying the foundation for edge generation and path search.

As shown in Algorithm 1, the procedure consists of three stages: (1) sampling two collision points and testing their midpoint (Lines 4–6), (2) directional perturbation filtering to enforce single-channel traversability (Lines 7–17), and (3) DBSCAN-based clustering to extract representative centroids (Lines 20–21).

Algorithm 1 Improved Bridge Sampling

```

1: INPUT: state bounds  $\mathcal{B}$ , number of samples  $N$ , perturbation distance  $\delta$ , clustering parameters  $\epsilon, \text{minPts}$ 
2: OUTPUT: representative centroids  $C$ 
3:  $B \leftarrow \emptyset$ 
4: for  $i = 1$  to  $N$  do
5:   sample two points  $x, p \sim \text{Uniform}(\mathcal{B})$ 
6:   let  $m \leftarrow (x + p)/2$ 
7:   if  $m$  is collision-free then
8:     compute orthonormal directions  $\{d_k\}$ 
9:     let  $f \leftarrow 0$ 
10:    for each  $d_k$  do
11:      let  $m_k \leftarrow m + \delta d_k$ 
12:      if  $m_k$  is collision-free then
13:         $f \leftarrow f + 1$ 
14:      end if
15:    end for
16:    if  $f = 1$  then
17:       $B \leftarrow B \cup \{m\}$ 
18:    end if
19:  end if
20: end for
21:  $C \leftarrow \text{DBSCAN}(B, \epsilon, \text{minPts})$ 
22: return  $C$ 

```

3.2. Neural Local Segment Generation

Given the sparse beacon set C produced by Algorithm 1 (nodes of the planning graph), the planner constructs edges on demand by calling a local connector to generate feasible segments between selected beacon pairs. Concretely, Algorithm 2 implements $\text{LocalConnect}(c_i, c_j)$, and Algorithm 3 uses greedy expansion to decide which (c_i, c_j) to connect next.

Algorithm 2 Neural Local Segment Generator

```

1: INPUT: beacon pair  $(c_0, c_{\text{goal}})$ , environment latent  $Z$ , neural model  $\mathcal{N}$ , max steps  $T$ , threshold  $\epsilon$ 
2: OUTPUT: local segment  $\tau(c_0 \rightarrow c_{\text{goal}})$  or failure
3:  $c_{\text{curr}} \leftarrow c_0$ 
4: for  $t \leftarrow 1$  to  $T$  do
5:    $c_{\text{next}} \leftarrow \mathcal{N}(Z, c_{\text{curr}}, c_{\text{goal}})$ 
6:   if  $\|c_{\text{next}} - c_{\text{goal}}\| \leq \epsilon$  then
7:     append  $c_{\text{goal}}$  to  $\tau$ 
8:     return  $\tau$ 
9:   end if
10:   $\sigma \leftarrow \text{Interpolate}(c_{\text{curr}}, c_{\text{next}})$ 
11:  if  $\text{CollFree}(\sigma)$  then
12:    append all points in  $\sigma$  to  $\tau$ 
13:     $c_{\text{curr}} \leftarrow c_{\text{next}}$ 
14:  else
15:     $\rho \leftarrow \text{RRTConnect}(c_{\text{curr}}, c_{\text{next}})$ 
16:    if  $\rho \neq \text{failure}$  then
17:      append all points in  $\rho$  to  $\tau$ 
18:       $c_{\text{curr}} \leftarrow c_{\text{next}}$ 
19:    else
20:      BACKTRACK: remove last two points from  $\tau$ 
21:       $c_{\text{curr}} \leftarrow$  last point in  $\tau$ 
22:    end if
23:  end if
24: end for
25: return failure

```

Algorithm 3 Bridge-Guided Hybrid Planner (Overall Pipeline)

```

1: INPUT: start  $c_s$ , goal  $c_g$ , state bounds  $\mathcal{B}$ , obstacle map  $x_{\text{obs}}$ 
2: OUTPUT: feasible path  $\pi$  or failure
3:  $Z \leftarrow \text{EnvEncoder}(x_{\text{obs}}; \theta^e)$ 
4:  $C \leftarrow \text{Algorithm 1}(\mathcal{B}, N, \delta, \epsilon, \text{minPts})$  // bridge beacons (nodes)
5: initialize open set  $\mathcal{O} \leftarrow \{c_s\}$ , path  $\pi \leftarrow [c_s]$ 
6: while  $\mathcal{O} \neq \emptyset$  and budget not exhausted do
7:   select next beacon  $c_{\text{next}}$  from  $C$  by greedy cost (distance + goal residual)
8:    $\tau \leftarrow \text{Algorithm 2}(c_{\text{curr}}, c_{\text{next}}, Z, \mathcal{N}, T, \epsilon)$ 
9:   if  $\tau \neq \text{failure}$  then
10:    append  $\tau$  to  $\pi$ , set  $c_{\text{curr}} \leftarrow c_{\text{next}}$ 
11:   else
12:    backtrack / expand alternative beacon candidates
13:   end if
14:   if  $\|c_{\text{curr}} - c_g\| \leq \epsilon$  then
15:     return  $\pi$ 
16:   end if
17: end while
18: return failure

```

First, the environment map x_{obs} is encoded into a low-dimensional latent variable Z via the environment encoder:

$$Z = \text{EnvEncoder}(x_{\text{obs}}; \theta^e). \quad (1)$$

This latent Z captures multi-scale spatial features of the obstacle distribution through a multi-layer convolutional and pooling architecture.

Given the current configuration c_t and target configuration c_{goal} , the state predictor—a multi-layer perceptron (MLP) with ReLU activations—outputs the next-step configuration:

$$\hat{c}_{t+1} = \text{StatePredictor}(Z, c_t, c_{\text{goal}}; \theta^p). \quad (2)$$

During training, successful trajectories $\{c_0^*, c_1^*, \dots, c_T^*\}$ generated by RRT* planners serve as demonstrations. A sliding window of length τ is applied to each demonstration trajectory to form transition samples. Each sample consists of

$$(Z, c_t^*, c_{\text{goal}}^*, \{c_{t-1}^*\}) \quad (3)$$

as inputs, with the next-step configuration c_{t+1}^* as the supervised label.

The network parameters θ^p are learned by minimizing the mean squared error between predicted and true configurations:

$$\ell_{\text{path}} = \frac{1}{N_p} \sum_{j=1}^{N_p} \sum_{i=0}^{\tau-1} \|\hat{c}_{j,i+1}^* - c_{j,i+1}^*\|^2. \quad (4)$$

During inference, the model iteratively predicts $\hat{c}_{t+1}$ from $(Z, c_t, c_{\text{goal}})$ until $\|c_{t+1} - c_{\text{goal}}\| \leq \epsilon$, where ϵ is a distance threshold. Upon reaching the vicinity of the goal, the generation stops, yielding a trajectory segment $\{c_0, \dots, c_T\}$.

Each adjacent pair (c_i, c_{i+1}) is then validated: if a direct Steer-To connection is collision-free, it is kept; otherwise, RRT-Connect is invoked to repair that segment. By concatenating all successful partial segments, a globally feasible path from start to goal is efficiently constructed.

3.3. Greedy Expansion with a Distance–Goal Composite Cost

Once the bridge points have been clustered, the planner proceeds in a purely greedy fashion, but the candidate ordering is no longer determined by “nearest first” alone. Let

c_{curr} be the current configuration, c_{goal} the goal, $U \subseteq C$ the set of as-yet unvisited bridge centres, and $\mathcal{S}$ a LIFO stack that stores the committed waypoints ($\mathcal{S} = \emptyset$ at start-up). At every expansion cycle the planner queries the k geometrically closest points to c_{curr} in the set $U \cup \{c_{\text{goal}}\}$, denoted $\mathcal{N} = \{x_1, \dots, x_k\}$. For each candidate $x \in \mathcal{N}$, it evaluates the *composite cost*

$$f(c_{\text{curr}}, x) = \underbrace{\|c_{\text{curr}} - x\|}_{\text{local hop}} + \lambda \underbrace{\|x - c_{\text{goal}}\|}_{\text{goal residual}}, \quad (5)$$

where the weight $\lambda \in [0, 1]$ biases how eagerly the search moves towards the goal ($\lambda = 0$ yields the original “pure nearest” rule; $\lambda \approx 0.3$ proved robust in our experiments). The candidates are sorted in ascending f ; the planner then tests them in that order, invoking the local segment generator up to R_{max} times per candidate (each call limited by a run-time budget T_{max}). The first candidate that returns a collision-free path is accepted: the segment is spliced into the global solution, c_{curr} is updated to the newly reached point, that point is removed from U , and it is pushed onto $\mathcal{S}$. If every candidate fails, the algorithm backtracks one step by popping $\mathcal{S}$; exhaustion of the stack signals overall failure. Conversely, as soon as c_{curr} links directly to c_{goal} , the concatenated stack constitutes a complete solution path. Compared with the pure-nearest scheme, this composite-cost heuristic keeps the search lightweight while sharply reducing the likelihood of wandering into dead-end corridors, yielding millisecond-level expansions in all tested scenarios.

3.4. Probabilistic Completeness

The proposed planner is probabilistically complete, i.e., if there exists a collision-free path in the configuration space $\mathcal{C}_{\text{free}}$, then as the number of bridge-sampling iterations $N \rightarrow \infty$ and the number of local repair tries per segment $R_{\text{max}} \rightarrow \infty$, the probability that the algorithm finds a solution approaches one.

Let π^* be any collision-free path from the start q_{start} to the goal q_{goal} . Because $\mathcal{C}_{\text{free}}$ is an open set, there exists a finite sequence of waypoints

$$q_{\text{start}} = w_0, w_1, \dots, w_K = q_{\text{goal}} \quad (6)$$

such that straight-line connections between consecutive w_i lie entirely in $\mathcal{C}_{\text{free}}$. We will show that the planner will almost surely discover a sequence of bridge-points clustered near these waypoints and successfully connect them.

- **Sampling of Bridge Points.** Each bridge-sampling iteration samples two collision configurations in $\mathcal{C}_{\text{obs}}$ and retains their midpoint m if it lies in $\mathcal{C}_{\text{free}}$ and passes the directional check. As $N \rightarrow \infty$, the standard Bridge Test is known to generate points whose density converges to that of the medial axis of $\mathcal{C}_{\text{free}}$. The added directional-filter retains only those midpoints lying in narrow passages, but does not prevent sampling arbitrarily close to any given narrow passage segment. Hence for each subsegment $[w_{i-1}, w_i]$, there is a nonzero probability that some sampled midpoint falls within an ϵ -ball around it. By the law of large numbers, with probability one, every ϵ -ball around each w_i will eventually contain at least one bridge point.
- **Density-Based Clustering.** DBSCAN with fixed ϵ and minPts clusters these midpoints. Since infinitely many samples appear in each ϵ -neighborhood of w_i , each neighborhood forms a cluster whose centroid lies within distance ϵ of w_i .
- **Local Repair Completeness.** Between each pair of neighboring centroids c_{i-1} and c_i , we invoke up to R_{max} attempts of the local segment generator, falling back on RRT-Connect if needed. RRT-Connect itself is probabilistically complete: as the number of samples (here local repair attempts) goes to infinity, it will find a collision-free connection between any two points in the same connected component of $\mathcal{C}_{\text{free}}$ with

probability one. Therefore, for sufficiently large $R_{\max}$, the probability of failing to join c_{i-1} to c_i tends to zero.

Combining these facts, for any fixed finite sequence of waypoints along π^* , the probability that (i) each vicinity of w_i contains a bridge-cluster centroid and (ii) each pair of consecutive centroids is successfully connected tends to one as $N, R_{\max} \rightarrow \infty$. Hence the overall probability that the planner returns a complete collision-free path approaches one.

4. Experimental Results

4.1. Implementation Details

This work is implemented in MATLAB R2024a (MathWorks, Natick, MA, USA) using the Deep Learning Toolbox. All experiments are conducted on a workstation with an Intel i5-13400 CPU (Intel, Santa Clara, CA, USA), an NVIDIA RTX 4070 GPU, 32GB RAM (Nvidia, Santa Clara, CA, USA), and Windows 10 operating system.

Neural Network Architecture: We design a 4-layer multilayer perceptron (MLP) as the segment generator. The network input includes the current configuration, the goal configuration, and an environment encoding, while the output is a fixed-length trajectory segment. For 2D tasks, the environment is encoded as a flattened occupancy grid directly fed into the MLP. For 3D tasks, we use a convolutional neural network (CNN) to extract spatial features from voxel maps, which are then concatenated to the planner input.

Training Data: Separate datasets are constructed for different task domains:

- **2D navigation:** We generate 500 maze-like maps with narrow passages (each of size 100×100), and sample 200 random start-goal pairs per map. Paths are computed using RRT*, yielding 100,000 total training trajectories.
- **3D spatial planning:** We construct 10 voxel environments ($5\text{ m} \times 5\text{ m} \times 2.5\text{ m}$) with double-hole walls and central obstacles. Each map provides 1000 RRT*-planned paths, totaling 10,000 trajectories.

Each of the above datasets is used to train a separate model. All test-time queries use unseen start-goal pairs to evaluate generalization.

Bridge Point Generation: We apply centroid-perturbed bridge sampling in free space. For each candidate obstacle pair, midpoints are sampled and validated via local collision checks to retain only those between real obstacle gaps. Valid bridge points are then projected back to high-dimensional configuration space using inverse kinematics and further filtered by reachability and collision constraints. Finally, DBSCAN clustering is used (with radius 3–5% of environment scale and minimum samples of 4) to extract centroid nodes for sparse graph construction.

Evaluation Settings: All planners are tested under a unified evaluation framework. Sampling-based methods are capped at 10,000 iterations. Learning-based planners are allowed to generate up to 150 neural proposals, and each RRT-Connect repair attempt is bounded by 5000 iterations.

4.2. Results

To evaluate the adaptability and effectiveness of the proposed path planning framework, we conduct comprehensive experiments across three representative task domains: 2D navigation, 3D volumetric motion, and high-DOF robotic manipulation. All methods are executed on a unified hardware platform under equal computational budgets.

(1) **2D Navigation Tasks:** We consider two types of randomly generated maze-like maps (Map1 and Map2), as shown in Figure 3, each sized at 100×100 grid cells, containing 3 and 6 narrow corridors, respectively. In Map1, although all planners succeed in some trials, sampling-based methods suffer from low success rates due to inefficient sampling in

narrow regions (e.g., RRT achieves only 6.67%). MPNet demonstrates relatively shorter path lengths and lower planning times due to prior learning but struggles in Map2, where the increased number of narrow corridors causes predicted waypoints to miss passage entrances, leading to frequent fallback to classical planners. This significantly increases planning time, as observed in Map1 where MPNet produces a sample that crosses through a wall, necessitating full replanning. In contrast, our method traverses open regions efficiently while explicitly targeting passage entrances using bridge-point clusters. Under the tested budget and trial set, it solves all instances in both Map1 and Map2 (100% success), while also achieving the shortest average path length and the lowest planning time.

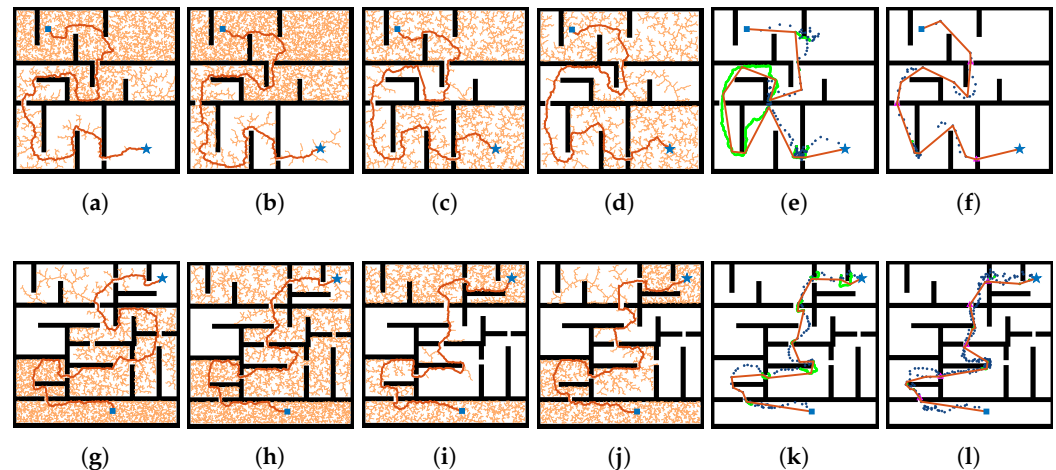


Figure 3. Qualitative comparison of six planners on two cluttered 2D workspaces. From left to right in each row: RRT, RRT*, BiRRT, RRT-Connect, MPNet, and the proposed method. (a–f) correspond to Map1; (g–l) to Map2. All planners were given the same computation budget. In the subfigures of MPNet and the proposed method, The blue square and star denote the start and goal configurations, the blue points denote the candidate outputs of the neural local segment generator, the green points represent the samples generated by the traditional planner, the triangles indicate the clustered bridge points, and the orange lines depict the final planned path.

(2) 3D Navigation Tasks: As shown in Table 1 and Figure 4, in the constructed 3D environment characterized by dual narrow holes and dual barrier constraints, traditional sampling-based planners exhibit significant limitations in feasibility and efficiency. The irregular distribution of free space and the presence of severely constrained passage geometries hinder effective exploration, leading to frequent planning failures or suboptimal paths, especially for methods that rely on random expansion without structural awareness. Although learning-based approaches such as MPNet improve overall feasibility, they tend to rely heavily on fallback classical planners when encountering bottleneck regions—particularly around narrow holes—resulting in longer planning times and unnecessarily convoluted trajectories. This fallback mechanism highlights the difficulty of generalizing across tight passages using purely data-driven strategies. In contrast, our method demonstrates superior adaptability and efficiency in such constrained topologies. By explicitly modeling passage geometry, the planner is able to rapidly identify and traverse critical narrow regions. This structure-aware guidance not only ensures full task completion but also results in shorter planning times and lower trajectory costs with minimal variance. The proposed approach thereby achieves a favorable balance between success rate, path optimality, and computational efficiency in highly constrained 3D planning scenarios. We note that the 100% success on Map3D is an empirical statistic over the tested instances under a fixed budget; it indicates that our topology-aware guidance and

completeness-preserving repair substantially improve entrance discovery and connectivity in the presence of multiple tight holes.

Table 1. Comparison of planners.

Maps	Algorithm	Time (s)	Path Length	Success Rate
Map1	RRT	6.71 ± 0.57	326.08 ± 7.25	6.67%
	RRT*	11.44 ± 0.51	307.80 ± 30.01	20.00%
	BiRRT	2.83 ± 0.62	315.11 ± 24.85	53.33%
	RRT-Connect	4.08 ± 0.61	334.52 ± 31.12	46.67%
	MPNet	3.75 ± 0.57	276.31 ± 14.79	73.33%
	Ours	1.58 ± 0.16	240.51 ± 8.26	100.00%
Map2	RRT	4.48 ± 0.80	452.15 ± 35.10	3.33%
	RRT*	10.10 ± 1.10	441.30 ± 28.50	16.67%
	BiRRT	3.58 ± 0.75	470.22 ± 40.12	33.33%
	RRT-Connect	4.35 ± 0.90	463.50 ± 36.00	40.00%
	MPNet	3.10 ± 0.65	350.44 ± 20.30	83.33%
	Ours	2.15 ± 0.20	345.10 ± 15.80	100.00%
Map3D	RRT	5.38 ± 0.95	23.85 ± 2.11	23.33%
	RRT*	20.21 ± 1.20	22.20 ± 1.72	40.00%
	BiRRT	4.77 ± 0.88	25.28 ± 2.26	53.33%
	RRT-Connect	6.50 ± 0.92	26.51 ± 2.14	50.00%
	MPNet	3.85 ± 0.70	19.04 ± 1.28	93.33%
	Ours	3.20 ± 0.30	18.83 ± 1.14	100.00%

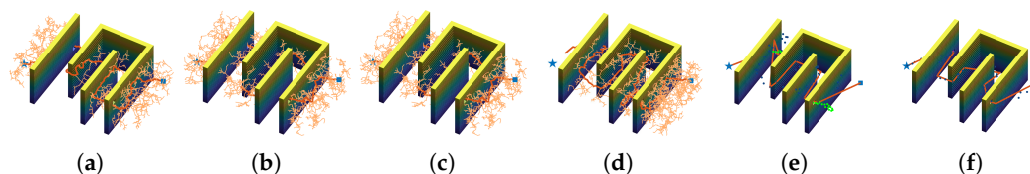


Figure 4. Qualitative comparison of six planners on the 3D workspace. From left to right: (a) BiRRT, (b) RRT-Connect, (c) RRT*, (d) RRT, (e) MPNet, (f) proposed method. All planners share the same start and goal configurations and identical computation budgets.

(3) Parameter Sensitivity: We conduct a controlled study on a representative 2D map by varying four key hyper-parameters in our framework: the bridge perturbation distance δ , the DBSCAN neighborhood radius ϵ , the number of bridge sampling trials N , and the maximum rollout steps T in the neural local connector. Figure 5 reports the success rate and planning time (mean $\pm$ standard deviation over repeated runs) under different parameter settings.

The perturbation distance δ governs the single-channel filtering in the improved bridge sampling (Algorithm 1). When δ is too small, the perturbations fail to sufficiently probe the local passage geometry, leading to less reliable bottleneck evidence and decreased success rates; when δ is too large, the perturbations may step outside the true corridor thickness and reject valid midpoints, which also degrades performance. As shown in Figure 5a, the best trade-off is achieved around $\delta = 0.5$, where the planner attains consistently high success while keeping the planning time low.

The parameter ϵ controls the clustering granularity of candidate bridge points. A very small ϵ tends to fragment a corridor into many small clusters, increasing the number of beacons and local connections, thus raising runtime; conversely, overly large ϵ may merge distinct bottlenecks and produce an over-sparse abstraction, weakening guidance and reducing success. Figure 5b suggests that $\epsilon \in [0.5, 1]$ yields the most stable behavior, balancing sparsity and representativeness of the passage centroids.

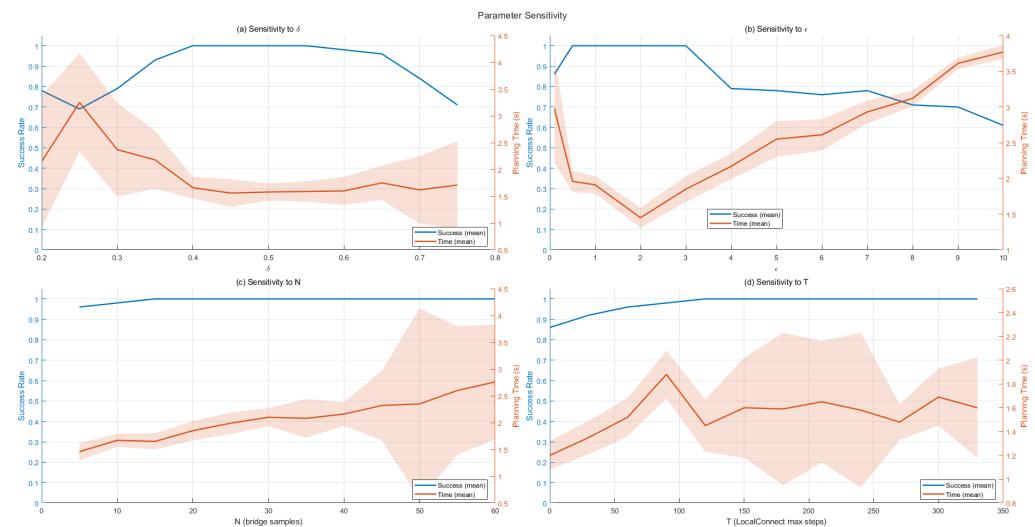


Figure 5. Parameter sensitivity on a representative 2D map. We vary the bridge perturbation distance δ , DBSCAN neighborhood radius ϵ , the number of bridge trials N , and the maximum rollout steps T in the local connector. The curves report success rate and planning time, where the orange shaded bands indicate ± 1 standard deviation over repeated runs.

The number of bridge trials N allocates computation to topology exposure. Increasing N generally improves the probability of discovering passage entrances, but also increases sampling/validation cost. In Figure 5c, success saturates quickly as N increases, while the planning time grows with larger N . Therefore, we set $N = 20$ as a practical knee point that preserves near-saturated success with moderate runtime overhead.

The rollout limit T bounds the horizon of the neural local segment generator (Algorithm 2). A small T may prematurely stop the connector before reaching the next beacon, causing additional repairs or failures; larger T increases the worst-case inference and checking cost. As shown in Figure 5d, $T = 150$ provides a robust success-time trade-off. Notably, our connector terminates early once a feasible connection is found (i.e., the rollout stops when the goal vicinity is reached), so increasing T beyond this value has marginal impact on the final performance in practice.

Based on the above sensitivity analysis, we use $\delta = 0.5$, $\epsilon \in [0.5, 1]$, $N = 20$, and $T = 150$ as default hyper-parameters in the remaining experiments, as they provide a stable balance between success rate and planning time on the evaluated 2D tasks.

5. Application Case: Steam Generator Plugging Task

5.1. Task Description

The plugging task takes place inside the steam generator of a nuclear power plant, which is a sealed cylindrical vessel with complex internal structures and severely limited working space. The interior is only accessible through a narrow maintenance port. As shown in Figure 6, during scheduled maintenance, a 12-degree-of-freedom robotic manipulator is required to carry a plugging device through the manhole, navigate past the water chamber section, and reach the main pipe region. The plugging operation is executed precisely at a position 200 mm in front of the pipe opening, ensuring the isolation of the steam generator from the primary reactor loop. Throughout this process, the robot must satisfy strict joint limit and collision avoidance constraints. The task constitutes a high-dimensional motion planning problem with pronounced spatial bottlenecks. In such constrained environments, conventional sampling-based planners often fail to return feasible solutions within a reasonable time budget.

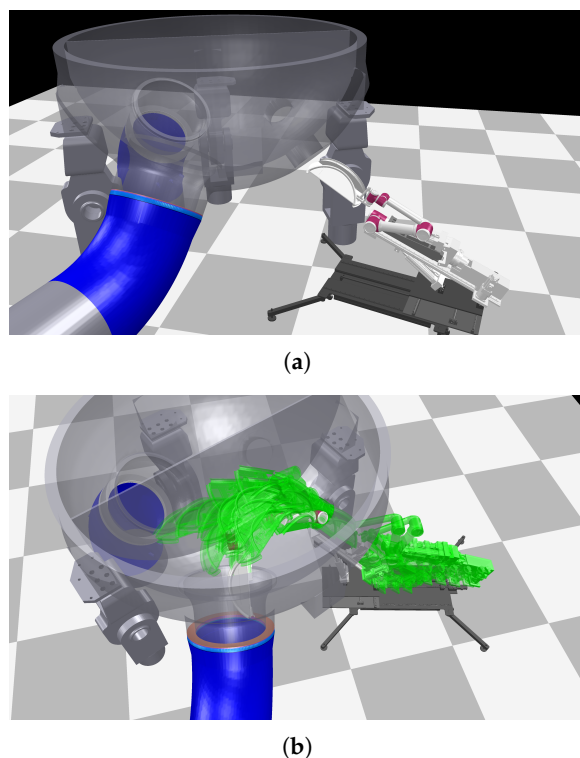


Figure 6. Demonstration in the steam-generator maintenance scenario. (a) Photorealistic layout of the steam generator with the 12-DOF plugging robot poised at the manway entrance. (b) Time-lapse rendering of the trajectory generated by the proposed bridge-guided planner.

5.2. Training Data

To support this application, we construct a specialized dataset reflecting the spatial and kinematic constraints of the real deployment scenario. Specifically, we simulate a confined cylindrical workspace resembling the interior of a nuclear steam generator, incorporating narrow access ports and curved wall boundaries. Within this setup, we randomly sample 2000 start-goal configurations of the 12-DOF robotic manipulator. For each configuration, a reference trajectory is generated using RRT*, forming a domain-specific training dataset designed to capture the complexity of constrained insertion and alignment maneuvers.

5.3. Evaluation

We assess planner performance in the highly constrained steam-generator environment, which imposes tight pose and orientation requirements on a 12-DOF robotic arm. Quantitative results are summarized in Table 2. Under the fixed time budget used in our evaluation, classical sampling-based planners (RRT, RRT*, BiRRT, RRT-Connect) fail to return a complete feasible solution, which reflects the long-horizon difficulty of exploring a high-dimensional narrow-topology space under runtime constraints (rather than the absence of feasible solutions in principle). MPNet achieves a success rate of 68.6% and a mean path length of 7.67 m. However, it frequently violates goal tolerances or collision constraints near the bottleneck, which triggers repeated replanning/repair and leads to a long average runtime of 243 s. In contrast, our bridge-guided framework improves the success rate to 80.6% and reduces the mean path length to 7.09 m, while achieving an average runtime of 210 s. We note that the gain in runtime and path length over MPNet is moderate in this application case. This is expected because (i) our topology-exposure stage performs additional collision/IK validation for each bridge-test candidate, whose overhead partially offsets the benefit of faster neural local connections; and (ii) when MPNet succeeds, its repaired solution is already close to feasible, leaving limited room

for further shortening the final path. Importantly, the main benefit of our method in this confined 12-DOF setting is improved robustness at the narrow-passage entrance and near the goal constraints, which reduces the frequency of costly replanning loops and yields a higher success rate under the same budget. These results suggest that while bridge-guided topology exposure remains effective in task-specific, high-DOF settings, further efficiency gains may require incorporating task-aware heuristics or learned IK-feasibility priors to reduce validation overhead.

Table 2. Results in the Steam Generator Scenario.

Algorithm	Time (s)	Path Length	Success Rate
RRT	—	—	0.00%
RRT*	—	—	0.00%
BiRRT	—	—	0.00%
RRT-Connect	—	—	0.00%
MPNet	243.24	7.67	70.00%
Ours	210.35	7.09	86.67%

5.4. Scalability to High-DOF Systems

As DOF increases, the dominant cost typically comes from geometric validation (collision checking and IK feasibility) rather than neural inference. Our method mitigates wasted exploration by factorizing long-horizon planning into (i) bridge-guided topology exposure that concentrates samples on bottleneck evidence and yields a sparse set of passage beacons (via DBSCAN), and (ii) fast local connections generated by a lightweight neural segment model. When the learned connector fails in tight regions, bounded RRT-Connect backtracking serves as a safety net, preserving feasibility and preventing catastrophic failures. Therefore, the scalability is mainly limited by the number of validation queries (bridge filtering and segment checking) times the per-query collision/IK cost, which grows with DOF and environment complexity. In future work, we will reduce this overhead by incorporating task-aware heuristics or learned IK-feasibility priors and by adopting hierarchical planning/refinement for very high-DOF systems.

6. Conclusions

This paper introduced a bridge-guided hybrid planner that combines an improved Bridge Test, DBSCAN-based clustering, and a learning-driven local planner to address motion planning for high-DOF robots in narrow passages. The proposed orthogonal-perturbation Bridge Test yields a sparser and more accurate set of passage samples, while the neural local planner backed by RRT-Connect repair achieves probabilistic completeness and competitive speed. Experiments on 2D and 3D benchmarks, as well as a 12-DOF steam generator scenario, demonstrated an 80.6% success rate and a 7.6% reduction in path length over MPNet, although the time advantage is partly offset by inverse-kinematics (IK) checks during bridge sampling, especially in sparsely obstructed environments.

Author Contributions: Conceptualization, J.L. and S.D.; Methodology, J.L.; Software, J.L.; Validation, J.L.; Formal analysis, J.L.; Investigation, J.L.; Resources, S.D.; Data curation, J.L.; Writing—original draft, J.L.; Writing—review and editing, S.D., G.X. and X.Y.; Supervision, S.D., G.X. and X.Y.; Project administration, S.D.; Funding acquisition, S.D. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Natural Science Foundation of China under Grants 62403338 and 62403340.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Data are available from the corresponding author upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. LaValle, S. *Rapidly-Exploring Random Trees: A New Tool for Path Planning*; Research Report 9811; Department of Computer Science, Iowa State University: Ames, IA, USA, 1998.
2. Karaman, S.; Frazzoli, E. Sampling-based algorithms for optimal motion planning. *Int. J. Robot. Res.* **2011**, *30*, 846–894. [[CrossRef](#)]
3. Kuffner, J.J.; LaValle, S.M. RRT-connect: An efficient approach to single-query path planning. In Proceedings of the 2000 IEEE International Conference on Robotics and Automation (ICRA), San Francisco, CA, USA, 24–28 April 2000.
4. Qureshi, A.H.; Miao, Y.; Simeonov, A.; Yip, M.C. Motion planning networks: Bridging the gap between learning-based and classical motion planners. *IEEE Trans. Robot.* **2020**, *37*, 48–66. [[CrossRef](#)]
5. Kavraki, L.E.; Svestka, P.; Latombe, J.-C.; Overmars, M.H. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE Trans. Robot. Autom.* **1996**, *12*, 566–580. [[CrossRef](#)]
6. Hsu, D.; Latombe, J.-C.; Motwani, R. Path planning in expansive configuration spaces. In Proceedings of the 1997 IEEE International Conference on Robotics and Automation (ICRA), Albuquerque, NM, USA, 25–25 April 1997.
7. Szkandera, J.; Kolingerová, I.; Maňák, M. Narrow passage problem solution for motion planning. In Proceedings of the International Conference on Computational Science, Amsterdam, The Netherlands, 3–5 June 2020.
8. Huang, J.; Su, H.; Au, K.W.S. Passage-traversing optimal path planning with sampling-based algorithms. *arXiv* **2025**, arXiv:2506.23614.
9. Huang, Y.; Lee, H.-H. Adaptive informed RRT*: Asymptotically optimal path planning with elliptical sampling pools in narrow passages. *Int. J. Control Autom. Syst.* **2024**, *22*, 241–251. [[CrossRef](#)]
10. Hsu, D.; Jiang, T.; Reif, J.; Sun, Z. The bridge test for sampling narrow passages with probabilistic roadmap planners. In Proceedings of the 2003 IEEE International Conference on Robotics and Automation (ICRA), Taipei, Taiwan, 14–19 September 2003.
11. Yershova, A.; Jaillet, L.; Siméon, T.; LaValle, S.M. Dynamic-domain RRTs: Efficient exploration by controlling the sampling domain. In Proceedings of the 2005 IEEE International Conference on Robotics and Automation (ICRA), Barcelona, Spain, 18–22 April 2005.
12. Khanmohammadi, S.; Mahdizadeh, A. Density avoided sampling: An intelligent sampling technique for rapidly-exploring random trees. In Proceedings of the International Conference on Hybrid Intelligent Systems, Barcelona, Spain, 10–12 September 2008.
13. Rodriguez, S.; Thomas, S.; Pearce, R.; Amato, N.M. Resampl: A region-sensitive adaptive motion planner. In *Algorithmic Foundation of Robotics VII: Selected Contributions of the Seventh International Workshop on the Algorithmic Foundations of Robotics*; Springer: Berlin/Heidelberg, Germany, 2008.
14. Zhang, L.; Manocha, D. An efficient retraction-based RRT planner. In Proceedings of the 2008 IEEE International Conference on Robotics and Automation (ICRA), Pasadena, CA, USA, 19–23 May 2008.
15. Li, S.; Dantam, N.T. Sample-driven connectivity learning for motion planning in narrow passages. In Proceedings of the 2023 IEEE International Conference on Robotics and Automation (ICRA), London, UK, 29 May–2 June 2023.
16. Strandberg, M. Augmenting RRT-planners with local trees. In Proceedings of the 2004 IEEE International Conference on Robotics and Automation (ICRA), New Orleans, LA, USA, 26 April–1 May 2004.
17. Lai, T.; Ramos, F.; Francis, G. Balancing global exploration and local-connectivity exploitation with rapidly-exploring random disjointed-trees. In Proceedings of the 2019 IEEE International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019.
18. Lai, T.; Morere, P.; Ramos, F.; Francis, G. Bayesian local sampling-based planning. *IEEE Robot. Autom. Lett.* **2020**, *5*, 1954–1961. [[CrossRef](#)]
19. Rodriguez, S.; Tang, X.; Lien, J.-M.; Amato, N.M. An obstacle-based rapidly-exploring random tree. In Proceedings of the IEEE International Conference on Robotics and Automation, Orlando, FL, USA, 15–19 May 2006.
20. Dalibard, S.; Laumond, J.P. Linear dimensionality reduction in random motion planning. *Int. J. Robot. Res.* **2011**, *30*, 1461–1476. [[CrossRef](#)]
21. Lee, J.; Kwon, O.S.; Zhang, L.; Yoon, S.-e. SR-RRT: Selective retraction-based RRT planner. In Proceedings of the 2012 IEEE International Conference on Robotics and Automation (ICRA), Saint Paul, MN, USA, 14–18 May 2012.
22. Orthey, A.; Toussaint, M. Section patterns: Efficiently solving narrow passage problems in multilevel motion planning. *IEEE Trans. Robot.* **2021**, *37*, 1891–1905. [[CrossRef](#)]
23. Tu, H.; Deng, Y.; Li, Q.; Song, M.; Zheng, X. Improved RRT global path planning algorithm based on Bridge Test. *Robot. Auton. Syst.* **2024**, *171*, 104570. [[CrossRef](#)]

24. Pérez-Higueras, N.; Caballero, F.; Merino, L. Learning human-aware path planning with fully convolutional networks. In Proceedings of the 2018 IEEE international conference on robotics and automation (ICRA), Brisbane, QLD, Australia, 21–25 May 2018.
25. Ichter, B.; Schmerling, E.; Lee, T.-W.E.; Faust, A. Learned critical probabilistic roadmaps for robotic motion planning. In Proceedings of the 2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 31 May–31 August 2020.
26. Huang, Z.; Chen, H.; Pohovey, J.; Driggs-Campbell, K. Neural informed rrt*: Learning-based path planning with point cloud state representations under admissible ellipsoidal constraints. In Proceedings of the 2024 IEEE International Conference on Robotics and Automation (ICRA), Yokohama, Japan, 13–17 May 2024.
27. Wang, W.; Zuo, L.; Xu, X. A learning-based multi-RRT approach for robot path planning in narrow passages. *J. Intell. Robot. Syst.* **2018**, *90*, 81–100. [[CrossRef](#)]
28. Lee, M.; Lee, J.; Lee, D. Narrow Passage Path Planning using Collision Constraint Interpolation. *arXiv* **2024**, arXiv:2410.20697. [[CrossRef](#)]
29. Marcucci, T.; Nobel, P.; Tedrake, R.; Boyd, S. Fast path planning through large collections of safe boxes. *IEEE Trans. Robot.* **2024**, *40*, 3795–3811. [[CrossRef](#)]
30. Cao, M.; Mao, H.; Tang, X.; Sun, Y.; Chen, T. A novel RRT*-Connect algorithm for path planning on robotic arm collision avoidance. *Sci. Rep.* **2025**, *15*, 2836. [[CrossRef](#)]
31. Huang, C.-M.; Hsu, S.-H. Efficient path planning for a microrobot passing through environments with narrow passages. *Micromachines* **2022**, *13*, 1935. [[CrossRef](#)] [[PubMed](#)]
32. Ruan, S.; Poblete, K.L.; Wu, H.; Ma, Q.; Chirikjian, G.S. Efficient path planning in narrow passages for robots with ellipsoidal components. *IEEE Trans. Robot.* **2022**, *39*, 110–127. [[CrossRef](#)]
33. Feng, B.; Jiang, X.; Bi, Y. A Robot Motion Planning Approach Based on Adaptive Multi-Tree Sampling. In Proceedings of the ASME International Mechanical Engineering Congress and Exposition, New Orleans, LA, USA, 29 October–2 November 2023.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.